



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Best Bike Paths: Requirements Analysis and Specification Document (RASD)

Software Engineering II

Advisor: Prof. Rossi Matteo Giovanni

Students:

- Lorusso Luca (10831296)
- Martorano Gabriele (10822443)

Version: 1.0

Date: 23-12-2025

Content

Content

List of Figures

List of Tables

1. Introduction.....	1
1.1. Purpose.....	1
1.1.1. Goals.....	1
1.2. Scope.....	2
1.2.1. World Phenomena.....	3
1.2.2. Shared Phenomena.....	3
1.3. Definitions, Acronyms and Abbreviations.....	4
1.4. Revision History.....	5
1.5. Reference Documents.....	5
1.6. Document Structure.....	5
2. Overall Description.....	6
2.1. Product perspective.....	6
2.1.1. Scenario.....	6
2.1.2. Class Diagram.....	8
2.2. Product functions.....	9
2.3. User Characteristics.....	12
2.4. Assumptions, dependencies and constraints.....	13
2.4.1. Assumptions.....	13
2.4.2. Dependencies.....	14
2.4.3. Constraints.....	14
3. Specific Requirements.....	16
3.1. External Interface Requirements.....	16
3.1.1. User Interfaces.....	16
3.1.2. Hardware Interfaces.....	16
3.1.3. Software Interfaces.....	17
3.1.4. Communication Interfaces.....	17
3.2. Functional Requirements.....	17
3.2.1. Requirement List.....	17
3.2.2. Use Case Diagram.....	19
3.2.3. Use Case.....	20
3.2.4. Goals - Requirements Mapping.....	27
3.3. Performance Requirements.....	28
3.4. Design Constraints.....	29
3.4.1. Standards compliance.....	29
3.4.2. Hardware limitations.....	29
3.4.3. Any other constraint.....	29
3.5. Software System Attributes.....	29
3.5.1. Reliability.....	29

3.5.2. Availability.....	30
3.5.3. Security.....	30
3.5.4. Maintainability.....	30
3.5.5. Portability.....	30
4. Formal Analysis using Alloy.....	32
4.1. Alloy Model.....	32
4.2. Alloy Results.....	39
5. Effort spent.....	41
6. References.....	42
6.1. References.....	42
6.2. Used tools.....	42

List of Figures

- 2.1. High Level View - Class Diagram
- 2.2. Path Search & Visualization - Product Function
- 2.3. Account Creation and Login - Product Function
- 2.4. Trip Recording - Product Function
- 2.5. Manually Reporting of Path Information - Product Function
- 2.6. Automatic Reporting of Path Information - Product Function
- 2.7. Publication of Data - Product Function
- 2.8. Personal Data and Preferences Management - Product Function
- 2.9. Trip History - Product Function
- 3.1. Guest User Abilities - Use Case Diagram
- 3.2. Registered User Abilities - Use Case Diagram
- 3.3. Secure Login - Sequence Diagram
- 3.4. Start/Stop Trip Recording - Sequence Diagram
- 3.5. Review Auto-Detected Issues - Sequence Diagram
- 3.6. Search and View Paths - Sequence Diagram
- 3.7. Manual Data Contribution - Sequence Diagram
- 3.8. View Trip Details - Sequence Diagram
- 3.9. Publish Validated Information - Sequence Diagram
- 4.1. Report FSM - Alloy Model
- 4.2. AutoDetected State - Alloy Result
- 4.3. PendingValidation State - Alloy Result
- 4.4. Draft State - Alloy Result
- 4.5. Published State - Alloy Result
- 4.6. SearchRequest - Alloy Result

List of Tables

- 1.1. Goals table
- 1.2. World Phenomena table
- 1.3. Shared Phenomena table
- 1.4. Definitions, Acronyms and Abbreviations table
- 2.1. Domain Assumption table
- 2.2. Domain Dependencies table
- 2.3. Domain Constraints table
- 3.1. System Requirements table
- 3.2. Goals/Requirements Mapping table
- 5.1. Member Effort Spent table

1. Introduction

In modern cities, urban traffic congestion and the growing demand for sustainable mobility are driving more people to use bicycles for commuting and leisure. However, cyclists often face difficulties in identifying safe, efficient, and comfortable routes, as existing mapping applications rarely provide updated information on road conditions, surface quality, or obstacles along bike paths.

Best Bike Paths (BBP) addresses this problem by offering a platform that supports the cycling community in discovering, evaluating, and sharing information about the best routes for bicycles.

Through a combination of manual input and automatic detection from smartphone sensors, users can record their rides, report issues such as bumps or maintenance needs, and confirm detected events before publication.

The system then aggregates and ranks bike routes based on their overall condition, reliability, and suitability, promoting safer and more informed cycling in congested urban environments.

1.1. Purpose

This document defines the requirements, assumptions and models of the *Best Bike Paths (BBP)* system.

It serves as a reference for developers and stakeholders to ensure a shared understanding of the system's objectives, functionalities, and constraints.

1.1.1. Goals

Goal	Description
From a registered user perspective...	
G01	Allow the RU to create an account and log in securely to access personalized features.
G02	Allow the RU to start, pause, and stop the automated recording of a trip, collecting data from sensors to know when RU is biking and collect statistics.
G03	Allow the RU to store completed trips in his personal history, preserving route and statistics.
G04	Allow the RU to view trip statistics, such as total distance, duration, and average speed.
G05	Allow the RU to automatically enrich trip data with weather information (e.g., temperature, wind, conditions) obtained from an EWS.

Go6	Allow the RU to manually insert the path, street by street.
Go7	Allow the RU to manually insert information about the condition and the obstacles of specific streets in the bike paths (e.g., "optimal," "requires maintenance," etc.).
Go8	Allow the RU to activate automated information detection, letting BBP infer obstacles acquiring data from GPS, accelerometer and gyroscope.
Go9	Allow the RU to review and confirm or correct automatically detected obstacles before publication, ensuring reliability.
G10	Allow the RU to save recorded or inserted paths information (obstacles/status) as draft, and decide later whether to publish it.
G11	Allow the RU to publish validated paths information, obstacles/status of the streets (either manual or validated automated data), for the community to access.
G12	Allow the RU to search for bike paths between a chosen origin and destination.
G13	Allow the RU to view and compare multiple paths ranked by score based on path condition and effectiveness.
From a guest user perspective...	
G14	Allow the GU to search for bike paths between an origin and a destination without registering.
G15	Allow the GU to view ranked paths on a map, displaying their relative scores and public reports about conditions and obstacles.

Table 1.1: Goals table

1.2. Scope

BBP is a software system that supports users to create and route the best bike paths. The software allows registered users to record personal trips with various statistics, possibly enriched with meteorological information, and all users can visualize the information about possible paths between an origin and a destination. Moreover registered users can publish bike paths' information in both manual and automated mode. While the first needs the manually insertion of streets' name in the path and their status, the latter access to the device GPS, accelerometer and gyroscope to acquire data that must be confirmed or corrected by the user.

If more paths exist between two points, they are visualized based on their score. The path score is computed through an algorithm that evaluates the effectiveness and the status of the path.

1.2.1. World Phenomena

Phenomena	Description
WPo1	Cyclists undertake bike trips
WPo2	Cyclists assess the status of bike paths
WPo3	External services exist that monitor and provide meteorological information
WPo4	External services exist that provide and compute geographical information
WPo5	Users need a path between an origin and a destination point
WPo6	RU (cyclists) possess a mobile device with GPS functionality and sensors (accelerometer, gyroscope)
WPo7	Roads and bike paths have real physical characteristics (such as name, condition, and position).

Table 1.2: World Phenomena table

1.2.2. Shared Phenomena

Phenomena	Description	Controller	Observer
SPo1	The RU logs into the system	RU	BBP
SPo2	The RU starts, pauses/stops, and saves a trip recording.	RU	BBP
SPo3	BBP calculates and stores a trip's statistics (distance, average speed, etc.) through sensors	BBP	RU
SPo4	BBP retrieves relevant meteorological data from an EWS for a specific trip	BBP	EWS
SPo5	BBP retrieves map data and routes from an EMS for a specific search	BBP	EMS
SPo6	The RU views stored trip data	RU	BBP
SPo7	The RU manually enters the streets of the path and related informations (e.g., status, presence of obstacles)	RU	BBP
SPo8	The RU activates the automated mode to let BBP detect potential obstacles while biking during the recording of a trip	RU	BBP

SP09	BBP analyzes sensor readings and proposes possible anomalies or potholes to the user	BBP	RU
SP10	The RU confirms or corrects the automatically detected obstacles before publication	RU	BBP
SP11	The RU decides to make information about a path (obstacles/status of streets) publishable	RU	BBP
SP12	The user (RU or GU) specifies an origin and a destination point	RU or GU	BBP
SP13	BBP displays the most suitable path(s), ranked by score, on a map	BBP	RU or GU

Table 1.3: Shared Phenomena table

1.3. Definitions, Acronyms and Abbreviations

Acronym	Definition
API	Application Programming Interface
Ax	Assumption number x
BBP	Best Bike Paths
Cx	Constraint number x
Dx	Dependency number x
EMS	External Map Service
EWS	External Weather Service
GPS	Global Positioning System
GU	Guest User
HTTPS	Hypertext Transfer Protocol Secure
RASD	Requirements Analysis and Definition Specification
RU	Registered User
SPx	Shared Phenomena number x
UCx	Use Case number x
WPx	World Phenomena number x

Table 1.4: Definitions, Acronyms and Abbreviations table

1.4. Revision History

Version 1.0 (23-12-2025): Initial release.

1.5. Reference Documents

- Specification document: Assignment RDD AY 2025-2026
- Provided course materials and notes
- Alloy documentation
- www.quotidianosanita.it

1.6. Document Structure

1. Introduction:

This section presents the overall purpose and objectives of the *Best Bike Paths (BBP)* project, outlining its motivation and main goals. It also defines the project's scope through an analysis of the relevant real-world and shared phenomena. Key terms, acronyms, and abbreviations are introduced for clarity, together with a revision history and reference materials. The section ends with an outline of the document structure.

2. Overall Description:

This section provides a general view of the BBP system, describing its context, main functionalities, and scenarios of use. It also includes conceptual and behavioral domain models (such as class and state diagrams), details on user characteristics, and the assumptions, dependencies, or constraints that influence system design and implementation.

3. Specific Requirements:

This section deepens the overall description by specifying the system's external interfaces - user, hardware, software, and communication. It defines all functional requirements through use cases and activity diagrams, and describes performance targets, design constraints, and software quality attributes such as reliability, availability, security, maintainability, and portability.

4. Formal Analysis:

This section explains the main objectives of the formal modeling activity and presents the Alloy model used to verify the system's correctness. It outlines the key facts, assertions and results of the formal checks, emphasizing how these contribute to validating the logical consistency of BBP.

5. Effort Spent:

This section summarizes the amount of work done by each team member, specifying their individual contribution to the preparation of this RASD.

6. References:

This section lists all bibliographic references, documents, and materials cited or used in the development of the present specification.

2. Overall Description

2.1. Product perspective

2.1.1. Scenario

Scenario 1: Trip Recording with Automated Mode, Obstacle Detection and Validation

Mario, a Registered User (RU) and cyclist, decides to record his trip using the BBP app to contribute to the path inventory. After logging in, Mario navigates to the "Record Trip" section and selects "Start Recording" with the Automated Mode activated. As he cycles, the Mobile Client continuously collects data from the GPS, accelerometer, and gyroscope of the device and computes live trip information locally. BBP monitors Mario's speed and, inferring he is cycling, uses the motion sensors to detect potential anomalies.

Midway through the route, BBP registers an unusually high frequency of non-typical vibrations and flags a candidate obstacle (e.g., a pothole) that requires verification. At the end of his ride, Mario selects "Stop and Save Trip": the Mobile Client stores the collected trip data and synchronizes it to the backend for persistence (at trip end, or when connectivity is available). Before archiving the trip, BBP retrieves data from the External Weather Service (EWS) to associate the temperature and wind conditions with his performance.

Subsequently, BBP presents Mario with a trip summary and notifies him of the automatically detected candidate obstacle. Mario reviews the report, confirms that it was indeed a small pothole, adds a brief description, and selects "Confirm and Publish". The BBP system saves the trip in Mario's personal statistics and makes the validated obstacle available to the community.

Scenario 2: Manual Mode Trip Registration and Publishable Path Information

Chiara, a Registered User (RU), has just completed a bike trip. She opens the BBP app, logs in and goes to the "Manual Mode / Register Trip" section to archive the trip and contribute information to the community dataset.

BBP asks Chiara to manually register the followed bike path by inserting the ordered list of street names composing the route. While inserting them, Chiara also provides the status of the relevant streets/segments (e.g., "Optimal", "Requires Maintenance", ...), according to what she experienced during the ride. For one street segment, she sets the status to "Requires Maintenance" and adds a short note about the pavement wear.

During the same manual registration, Chiara also inserts a manual obstacle report encountered along the route: she selects the street segment where it occurred (among the inserted streets), chooses the obstacle type (e.g., "Debris/Blockage"), sets a severity level and writes a brief description.

At the end of the manual trip registration, BBP shows Chiara a recap of the inserted path information (streets + statuses + obstacles) and lets her decide whether to save it as draft or publish it. Chiara selects Publish, making the contributed information available to all users as public data associated with the corresponding path segments.

Scenario 3: Personal Statistics Review

Marco, the same user from Scenario 1, wants to review his performance. He accesses the "Trip History" section and selects his previously recorded trip. The BBP system displays a complete report: the map of the route, total distance, duration, average speed, and the details of the EWS-enriched information. He also views the summary of any reported issues (in this case, the confirmed pothole). Marco can thus compare the effectiveness and difficulty of the trip based on objective and environmental parameters.

Scenario 4: Automatic Detection Review - Rejection of False Positive

Sofia, a Registered User (RU), records a bike trip using Automated Mode. While she is riding, the Mobile Client acquires GPS, accelerometer, and gyroscope readings locally as part of the trip recording.

After Sofia stops and saves the trip, BBP stores the recorded data and runs the automatic detection analysis on the persisted trip data, correlating sensor patterns with the reconstructed route and mapping the resulting candidate events to one or more path segments.

BBP then prompts Sofia to review the detected candidate anomalies (e.g., potential potholes or surface issues). Sofia checks the suggested events and realizes they were caused by a cobblestone section rather than actual damage. She selects the flagged segment(s) and chooses "Reject" or "Insert the correct status".

Since the detected issue is not confirmed, BBP keeps the trip in Sofia's private history but does not publish the false positive to the public path inventory. As a result, the Path Score computation for that segment is not affected by incorrect information.

Scenario 5: Path Search, Comparison and Ranking Using Published Information

Giulia, a Guest User (GU), wants to reach the main train station by bike. She opens BBP and enters her current position as the Origin and the station as the Destination, then starts a search.

BBP retrieves from External Map Service (EMS) multiple feasible bike paths between the two points and evaluates them using the currently published community information. One of the shortest routes (Route A) includes the "Viale Torino" segment previously reported as "Requires Maintenance", so BBP penalizes its score. Another route (Route B) is slightly longer but crosses segments with better conditions and no severe obstacles, so it receives a higher score. A third route (Route C) is feasible but less

suitable for bikes (e.g., main roads with low compatibility), and BBP assigns it a low effectiveness score.

BBP presents the routes in a ranked list and visualizes them on the map, allowing Giulia to compare them quickly. Seeing that Route B is ranked highest due to better overall conditions and effectiveness, Giulia selects Route B for her trip.

2.1.2. Class Diagram

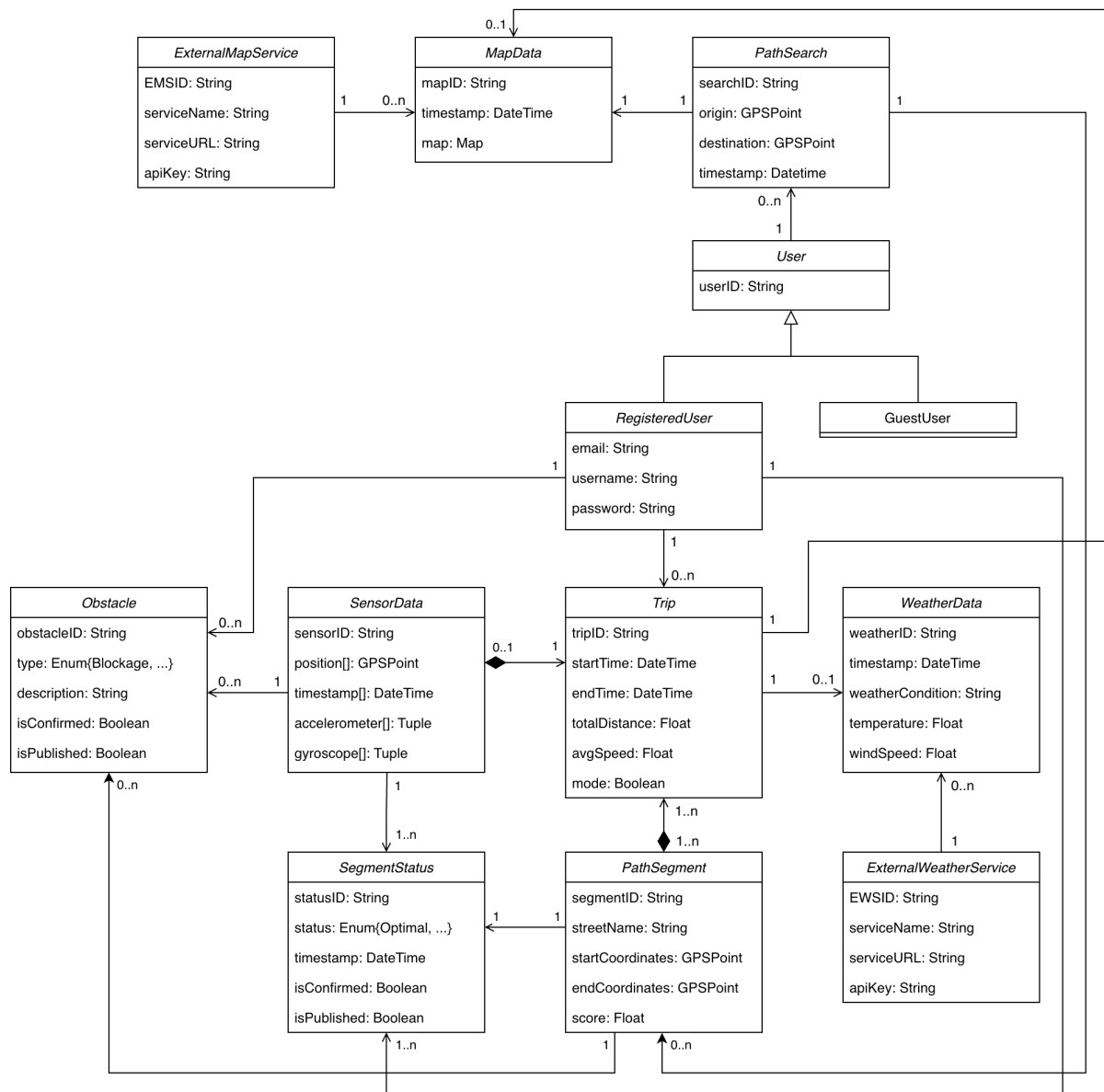


Figure 2.1: High Level View - Class Diagram

The Class Diagram illustrates the high-level data structure and the relationships between the key entities of the Best Bike Paths (BBP) system. The model is conceptually divided into three main areas: user management, raw trip data recording, and the path information lifecycle.

- **User Management and Access:** The abstract class User generalizes the system actors, specializing into RegisteredUser (who can contribute and record activities) and GuestUser (who can only consult published data). Any user type can initiate a path request, represented by the PathSearch class.
- **Trip Recording and Raw Data:** Each cycling activity is captured by a single Trip instance, which is tightly composed of multiple PathSegment instances. Data is deterministically enriched by WeatherData information (temperature, wind, conditions) obtained from a dedicated ExternalWeatherService. Data collection in automated mode is supported by the SensorData class (accelerometer and gyroscope), linked to the Trip instance.
- **Path Information Lifecycle and Evaluation:** Path knowledge is structured by SegmentStatus and Obstacles related to PathSegments. Registered users contribute to data quality by creating SegmentStatus (general status) or Obstacle (specific reporting of a problem like a pothole). Potential reports generated by SensorData are formalized into an Obstacle only after user confirmation (isConfirmed: Boolean).

2.2. Product functions

The following section describes the main functions of the system. The system is divided into three main Stakeholders: registered users, guest users and BBP.

Generic Functionalities

- **Path Search & Visualization:** users (RU or GU) can enter Origin and Destination (that must be valid, for example they need to be different) to obtain one, more or no bike paths on a map, based on the path scores. BBP asks the map to EMS, computes a path score (based on route effectiveness and path condition) and shows on the map ranked alternatives, or directly the only path found.

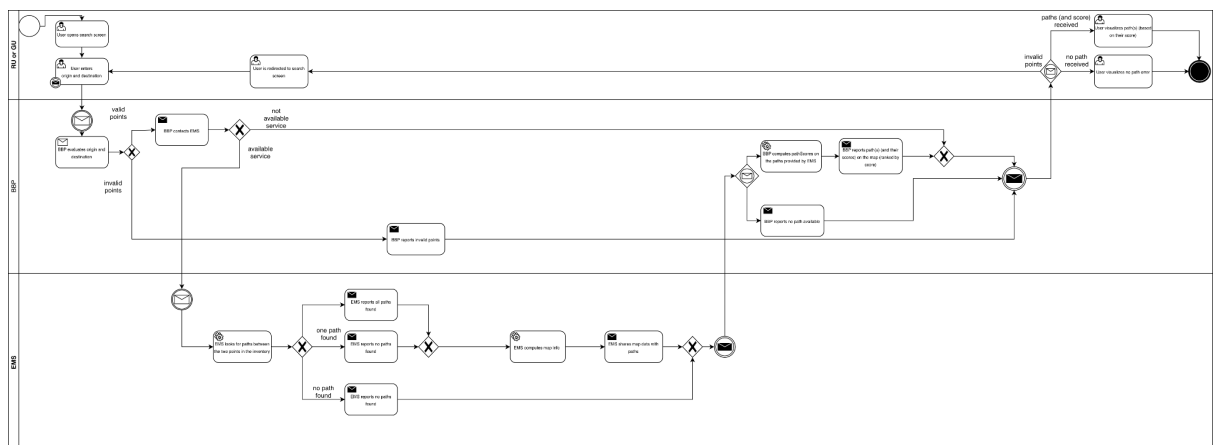


Figure 2.2: Path Search & Visualization - Product Function

Registered User Functionalities

- **Account Creation and Login:** registered users can create an account using email and password (specifying preferences such as permissions for automated mode)

and log in securely.

Authentication enables access to advanced features such as trip recording, reporting and statistics visualization.

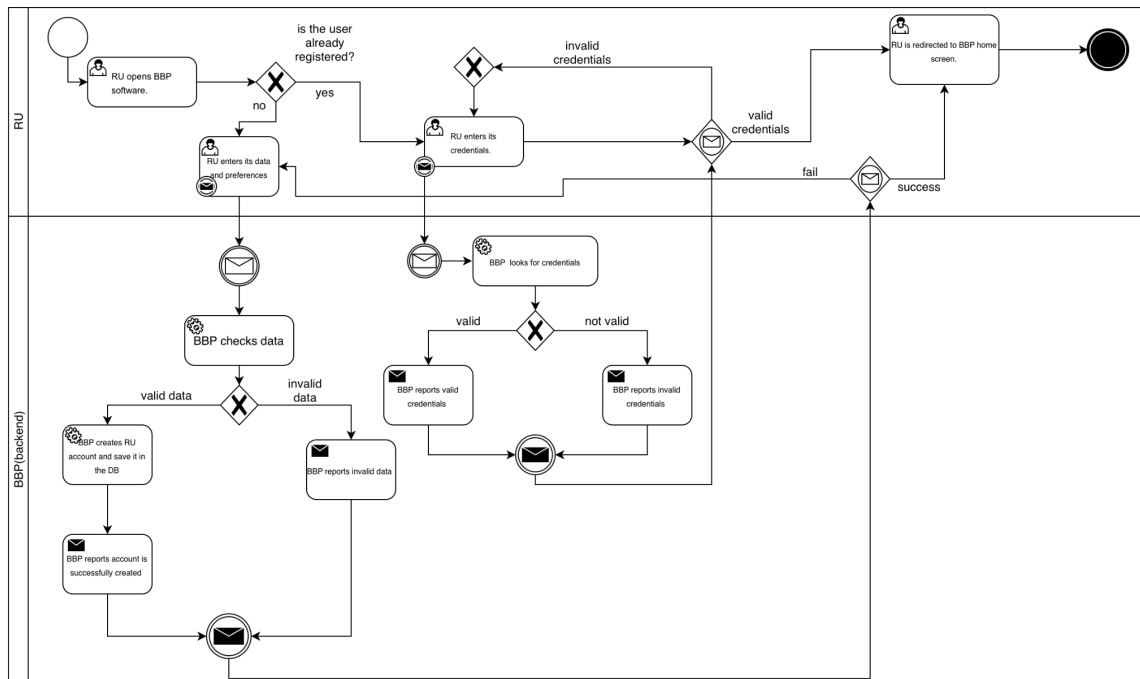


Figure 2.3: Account Creation and Login - Product Function

- Trip Recording:** BBP uses GPS, accelerometer and gyroscope sensors to detect total distance, average speed and other performance metrics and, when available, BBP also collects weather information through EWS. All trip information are stored in the user's personal trip history before the publish.

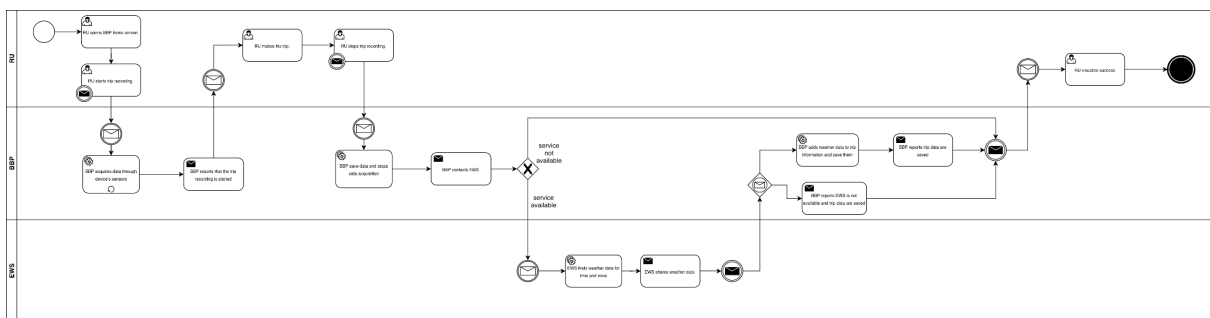


Figure 2.4: Trip Recording - Product Function

- Manually Reporting of Path Information:** the Registered User can manually insert a bike path (sequence of contiguous streets) including reports. This includes both: path status (*Optimal*, *Requires Maintenance*, etc.) and obstacle reports (with severity and short description). Submitted reports can be saved as drafts or immediately published to the community dataset.

- **Personal Data and Preferences Management:** users can edit their personal details (name, display preferences, measurement units) and adjust privacy settings related to trip visibility and publication.

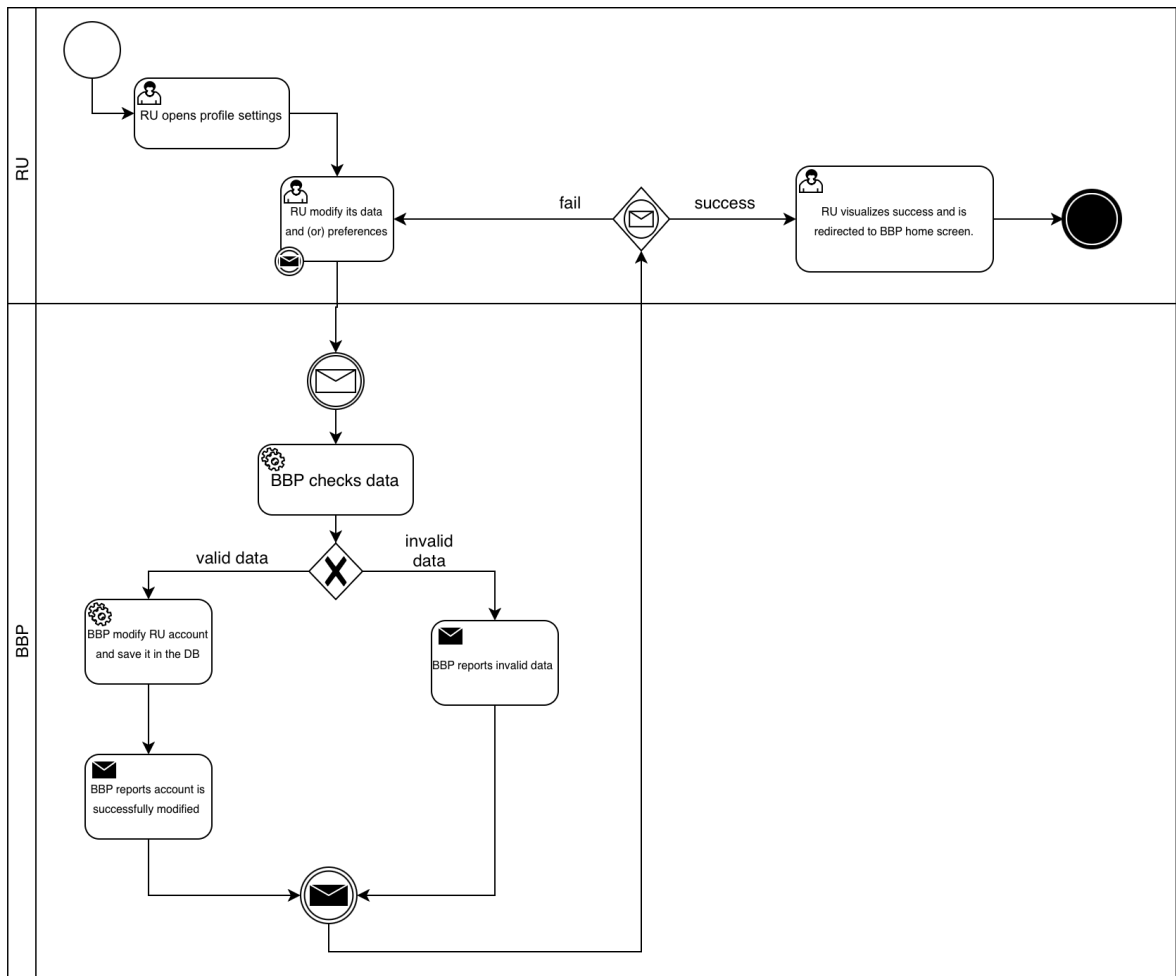


Figure 2.8: Personal Data and Preferences Management - Product Function

- **Trip History:** registered Users can access their *Trip History* section to review all past rides.

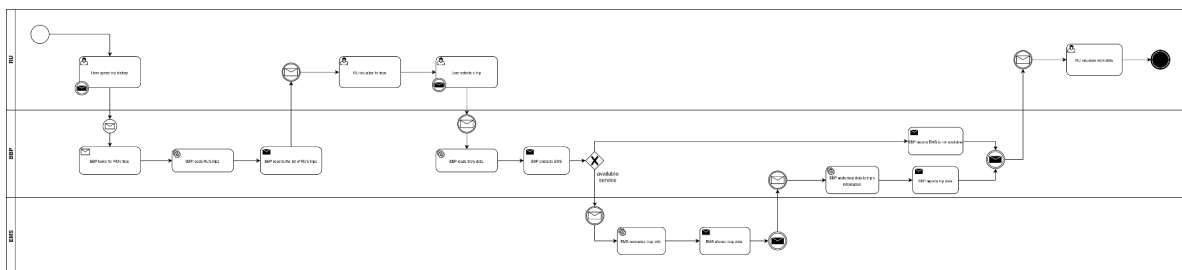


Figure 2.9: Trip History - Product Function

2.3. User Characteristics

The BBP system is designed to serve two primary, distinct categories of users: those who actively contribute data and those who consume published information for trip

planning. This section details the characteristics, motivation and competence level expected from each user type.

Registered Users (RUs): The Registered User is the core active participant in the BBP ecosystem. This user group is responsible for creating new content, maintaining the accuracy of the path inventory and utilizing personalized features.

- Primary Motivation and Goals: RUs seek to track and record their personal cycling performance (trips) and actively improve the community's path inventory by contributing verified data.
- Key Activities: Registered Users can start and stop automated trip recording, view and review their personal trips and related statistics (e.g., distance, speed, and EWS data). Before publication, they are required to review automatically detected issues and either confirm or correct them. Additionally, Registered Users can manually submit trip paths together with related conditions or specific obstacle reports.
- Technical Competence: Medium. RUs are expected to be familiar with using mobile applications, managing settings related to GPS, and operating the device's built-in sensors (accelerometer, gyroscope) for automated mode.
- Domain Competence: High. This group consists of cyclists whose direct experience is the foundation of the system's data quality. Their judgment is essential in validating potential false positives generated by the automated system, thereby ensuring the reliability of published reports.
- Access and Security: RUs require secure login credentials and personalized space to store private trip data.

Guest Users (GUs): The Guest User is the passive consumer of the information collected and validated by the community.

- Primary Motivation and Goals: GUs aim to quickly find the most suitable cycling path between a specified origin and destination, relying on the public reports and the system's generated path score.
- Key Activities: GUs interact primarily through search, map visualization, and consumption of public path condition summaries. They do not possess functionality to record trips or contribute new data.
- Technical Competence: Low. GUs are expected to be able to navigate a simple, map-based interface, similar to common navigation applications.
- Domain Competence: Variable. GUs may range from expert cyclists to casual users who rely heavily on the system's path score and ranking to make informed routing decisions.
- Access and Security: GUs access the system without authentication. The system must ensure they only interact with public data.

2.4. Assumptions, dependencies and constraints

2.4.1. Assumptions

Assumption	Description
A01	RU possess mobile devices (smartphones) that include functional

	GPS, accelerometer, and gyroscope sensors necessary for the Automated Mode.
A02	The community of users accepts that the path score is computed based on data provided by other users, trusting the system's ability to filter and validate contributions (e.g., the confirmation process in Automated Mode)
A03	Both RU and GU have a stable, reliable internet connection (mobile or Wi-Fi) sufficient to support data communication with the BBP system and map visualization
A04	An external, high-quality map rendering service (e.g., OpenStreetMap, Google Maps, or similar) is available for BBP to display paths and collect underlying geographical road data.

Table 2.1: Domain Assumptions table

2.4.2. Dependencies

Dependency	Description
D01	BBP requires a stable connection and subscription/API access to the External Weather Service to automatically enrich trip data with meteorological information (temperature, wind speed, etc.)
D02	BBP requires a stable connection and subscription/API access to the External Map Service to visualize paths on the map
D03	The system depends on the underlying mobile operating system (OS) and GPS hardware to accurately provide geographical coordinates, speed, and sensor data in real-time

Table 2.2: Domain Dependencies table

2.4.3. Constraints

Constraint	Description
C01	All collection, storage, and processing of personal data (e.g., trip history, location data) must comply with General Data Protection Regulation (GDPR) standards
C02	Data gathered through the Automated Mode must not be made public until it has been explicitly reviewed, confirmed, or corrected by the Registered User
C03	The user interface must be accessible through a mobile application running on common smartphones (Android/iOS), without requiring additional specialized hardware. The application must rely on the device's built-in sensors (GPS, accelerometer, gyroscope) and a standard Internet connection to provide the BBP functionalities.

C04	A manually inserted path must be representable as a contiguous sequence of adjacent streets.
-----	--

Table 2.3: Domain Constraints table

3. Specific Requirements

This chapter provides detailed specifications for the requirements previously mentioned that may require additional clarification for the development team's implementation.

3.1. External Interface Requirements

3.1.1. User Interfaces

The user interface of the Best Bike Paths (BBP) system is implemented as a mobile application, designed to run on smartphones carried by users during cycling activities. The interface is optimized for touch interaction and on-the-move usage, enabling users to easily record trips, visualize routes and statistics, and interact with map-based functionalities.

The interface supports:

- Real-time visualization of the recorded path and live basic trip indicators (e.g., elapsed time, current speed, distance estimate), while detailed statistics (e.g., final average speed and aggregated metrics) are computed and finalized at trip completion;
- interactive map exploration of bike paths, path scores and published reports;
- manual insertion of path conditions and obstacle reports;
- review, confirmation or correction of automatically detected issues before publication.

The mobile interface is designed to ensure usability in outdoor environments and during movement, minimizing user interaction complexity while cycling.

Additional auxiliary interfaces are provided to support profile creation and management and system feedback.

3.1.2. Hardware Interfaces

BBP interfaces directly with standard smartphone hardware to support its core functionalities. In particular, during the Automated Mode, the system relies on the mobile device's built-in sensors, including:

- Global Positioning System (GPS), used for real-time location tracking and route reconstruction;
- Accelerometer and Gyroscope, used for motion analysis and automatic detection of potential path anomalies.

These hardware components are accessed through the mobile operating system APIs and are required to be present and functional on the user's device for the Automated Mode to operate correctly.

No additional or specialized hardware is required beyond a standard smartphone equipped with the aforementioned sensors and an internet connection.

3.1.3. Software Interfaces

The system primarily functions as a standalone application. The core external dependencies are the EWS API required for enriching trip data and a Map Rendering API (e.g., GIS service) to display path geometries and user reports.

3.1.4. Communication Interfaces

The BBP system relies on HTTPS/TLS-secured communication for all data exchanges between the Mobile Client and the backend services. This includes the transmission of authentication data, trip information, sensor-derived reports and path search requests.

Communication between the backend and external services, the External Weather Service (EWS) and the External Map Service (EMS), is also performed exclusively over HTTPS, in compliance with security and data protection requirements.

All communication interfaces are designed to ensure confidentiality, integrity and authentication of exchanged data, in accordance with the system's security constraints.

3.2. Functional Requirements

3.2.1. Requirement List

Requirement	Description
R01	The system shall enable Registered Users (RUs) to securely create an account and log in
R02	The system shall enable RUs to start, pause, and stop trip recording, continuously collecting GPS data in real-time
R03	The system shall store completed trip data, including the precise route and calculated metrics, in the RU's private history
R04	The system shall calculate and display trip performance statistics (e.g., total distance, duration, average speed) upon trip completion
R05	The system shall automatically query the External Weather Service (EWS) to retrieve and associate, if available, meteorological data (temperature, wind, conditions) with the stored trip data
R06	In Manual Mode, the system shall allow RUs to define a path by providing an ordered sequence of streets/segments.
R07	In Manual Mode, if the ordered sequence of streets/segments inserted by the RU does not form a contiguous path, the system shall reject the submission and prompt the RU to correct it.

R08	The system shall allow RUs to manually report an obstacle (specifying its type, and a description) of the streets in the path.
R09	The system shall allow RUs to manually report the status of the streets in the path.
R10	The system shall support an Automated Mode where it activates sensor data acquisition (GPS, accelerometer, gyroscope) and infers the biking activity based on the user's speed
R11	The system shall process accelerometer and gyroscope data from the Automated Mode to automatically detect and flag potential path anomalies (e.g., potholes)
R12	The system shall require the RU to review, confirm, or correct automatically detected anomalies before they are validated or made public
R13	The system shall store user-generated path information (manual reports and confirmed automated detections) as draft until the RU explicitly publishes it.
R14	The system shall make finalized path information (manual or confirmed automated reports) publicly accessible only upon explicit command from the RU
R15	The system shall enable any user (RU or Guest) to search for bike paths by specifying an origin and a destination
R16	The system shall compute a path score for each feasible route, based on the published Path Status, Obstacle reports and path
R17	The system shall display multiple resultant paths, ranked clearly by their path score, on a map interface.
R18	The system shall allow Guest Users to perform all path search and visualization functions without requiring login
R19	The BBP System shall interface with an External Map Service (EMS) via a dedicated API to retrieve the geographical base layer (map tiles) for visualization of paths and reports and routing retrieval.

Table 3.1: System Requirements table

3.2.2. Use Case Diagram

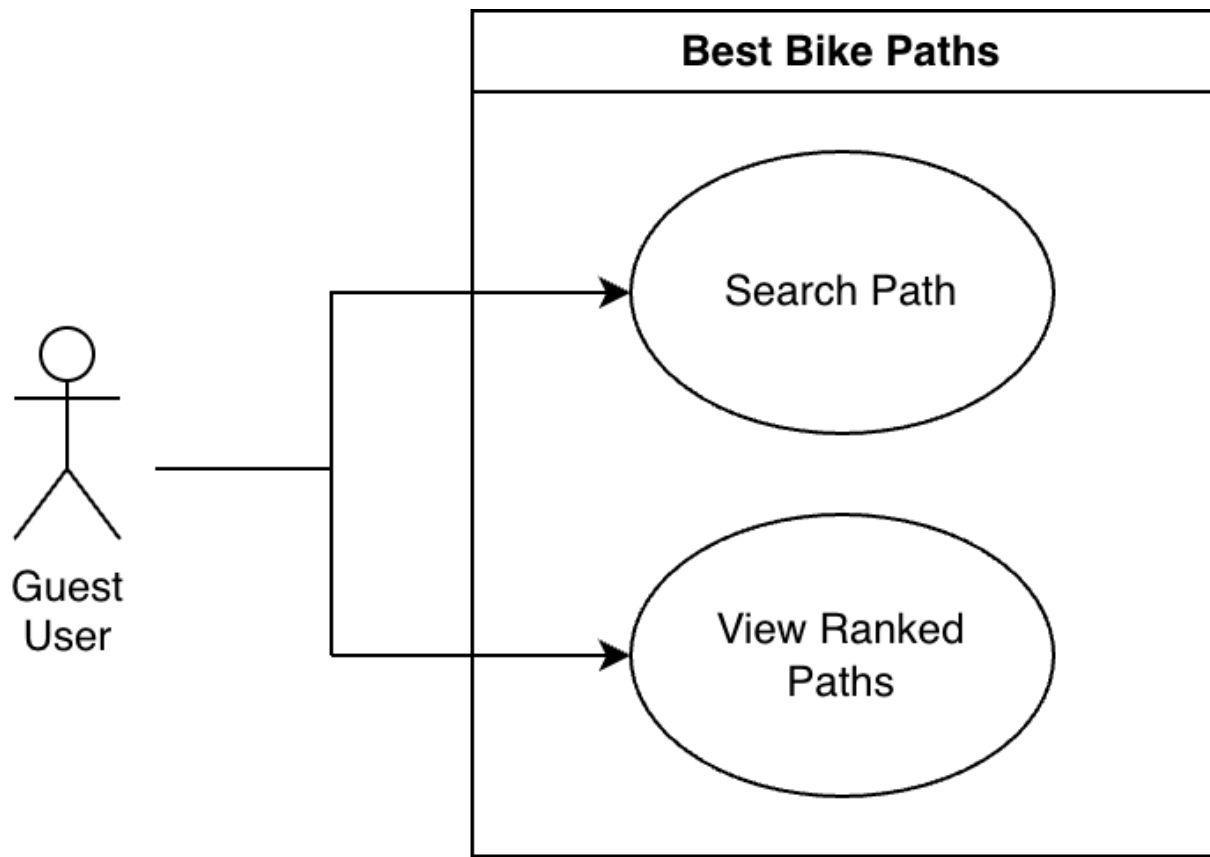


Figure 3.1: Guest User Abilities - Use Case Diagram

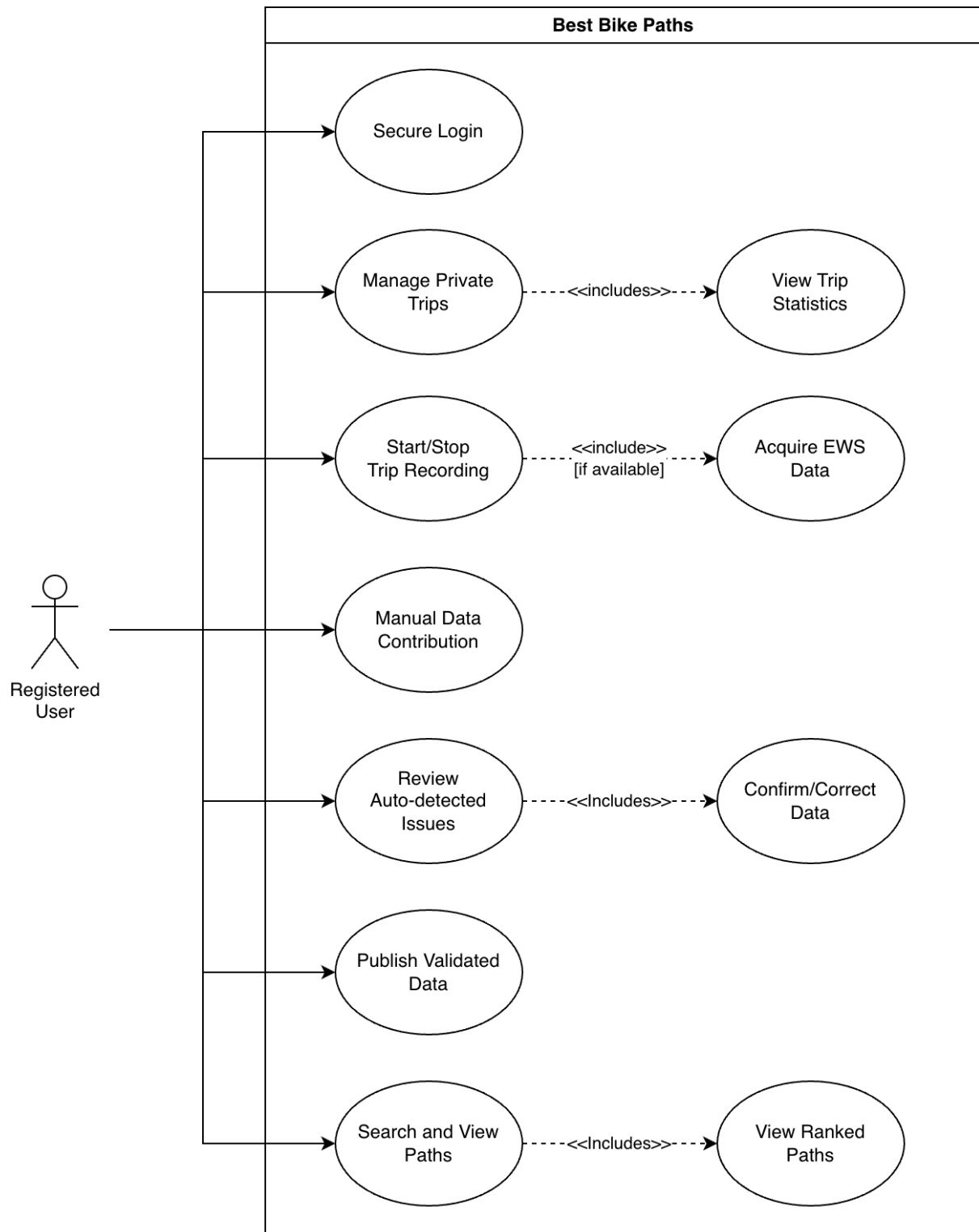


Figure 3.2: Registered User Abilities - Use Case Diagram

3.2.3. Use Case

In this section, the main use cases of the BBP system are described. Each use case specifies the involved actors, entry conditions, main flow of events, exit conditions, and possible exceptions.

The purpose of this section is to clarify how users and external services interact with the system to achieve the defined goals and satisfy the functional requirements. Use case diagrams and sequence diagrams are provided to illustrate the interactions between the actors and the system and the functions invoked during each process.

UC01: Secure Login

This use case establishes the foundation for all private interactions, ensuring that only authorized actors can access personal data.

Actors	Registered User (RU), BBP system, Database
Entry Conditions	The user is not authenticated in the system.
Flow of Events	1. RU requests access to the Login page. 2. BBP System sends the Login page. 3. RU enters credentials (email/password). 4. BBP System validates credentials against the Database. 5. BBP System issues a session token to the RU.
Exit Conditions	The Registered User is authenticated and redirected to their personal dashboard.
Exceptions	Invalid Credentials: The system displays an error message and requests a retry

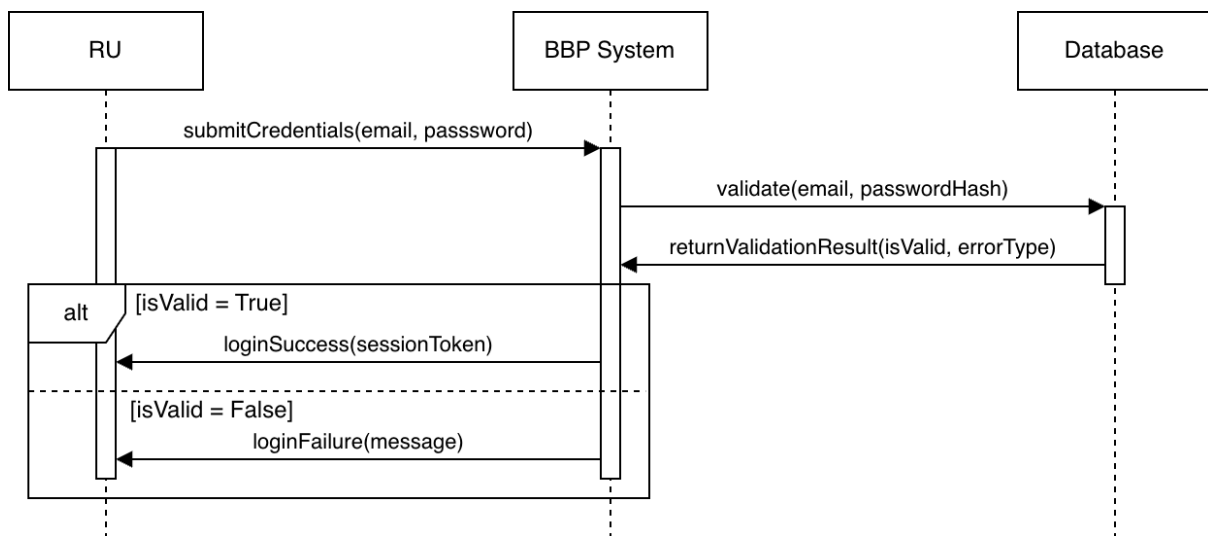


Figure 3.3: Secure Login - Sequence Diagram

UC02: Trip Recording and Auto Detect

This use case covers real-time data acquisition (mobile-side) and enrichment with external services, including the trigger for Automated Mode.

Actors	Registered User (RU), BBP System, Database, External Weather Service (EWS)
--------	--

Entry Conditions	RU is authenticated and on the trip recording page.
Flow of Events	1. RU selects "Start Recording", activating Automated Mode. 2. The device collects data. 3. RU selects "Stop Recording". 4. The device sends the collected data to the BBP system. 5. BBP System calculates trip statistics. 6. BBP System queries the EWS with the trip coordinates. 7. EWS, if available, replies with WeatherData. 8. BBP System archives trip data with WeatherData in the Database. 9. BBP System examines data to discover obstacles and status. 10. BBP System returns trip statistics and obstacles
Exit Conditions	The trip is archived in the RU's private history, enriched with weather data, obstacles and status. Then, RU will review auto-detected issues
Exceptions	EWS Failure: If the EWS service fails, the system archives the trip without weather data and notifies the RU

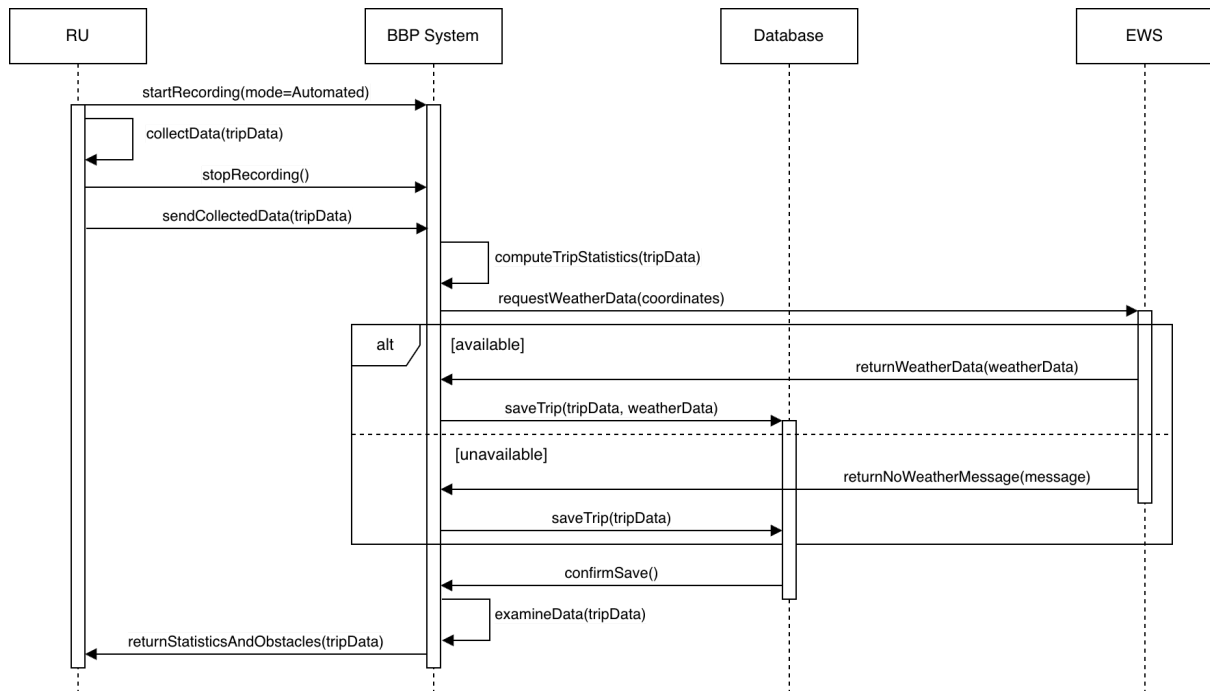


Figure 3.4: Start/Stop Trip Recording - Sequence Diagram

UC03: Review Auto-Detected Issues

This is the most critical and unique use case for your group's scope: the conditional validation of automatically generated data.

Actors	Registered User (RU), BBP System, Database
Entry Conditions	The trip is completed and the Database contains unconfirmed Obstacle (isConfirmed=False).

Flow of Events	1. RU accesses the Review page. 2. BBP System fetches unconfirmed reports from the Database. 3. BBP System displays raw reports and associated SensorData. 4. RU analyzes and selects Confirm or Correct/Reject. 5. BBP System sends the updates to the Database. 6. Database updates the isConfirmed field (if True, the data is now validated). 7. RU chooses to publish or draft the updated informations 8. Database updates the isPublished field (if True, the data is now available to the community).
Exit Conditions	The anomaly has been formally validated or rejected by the RU. If confirmed and published, the updated data is available to the community.
Exceptions	No Report Found: No reports found to validate

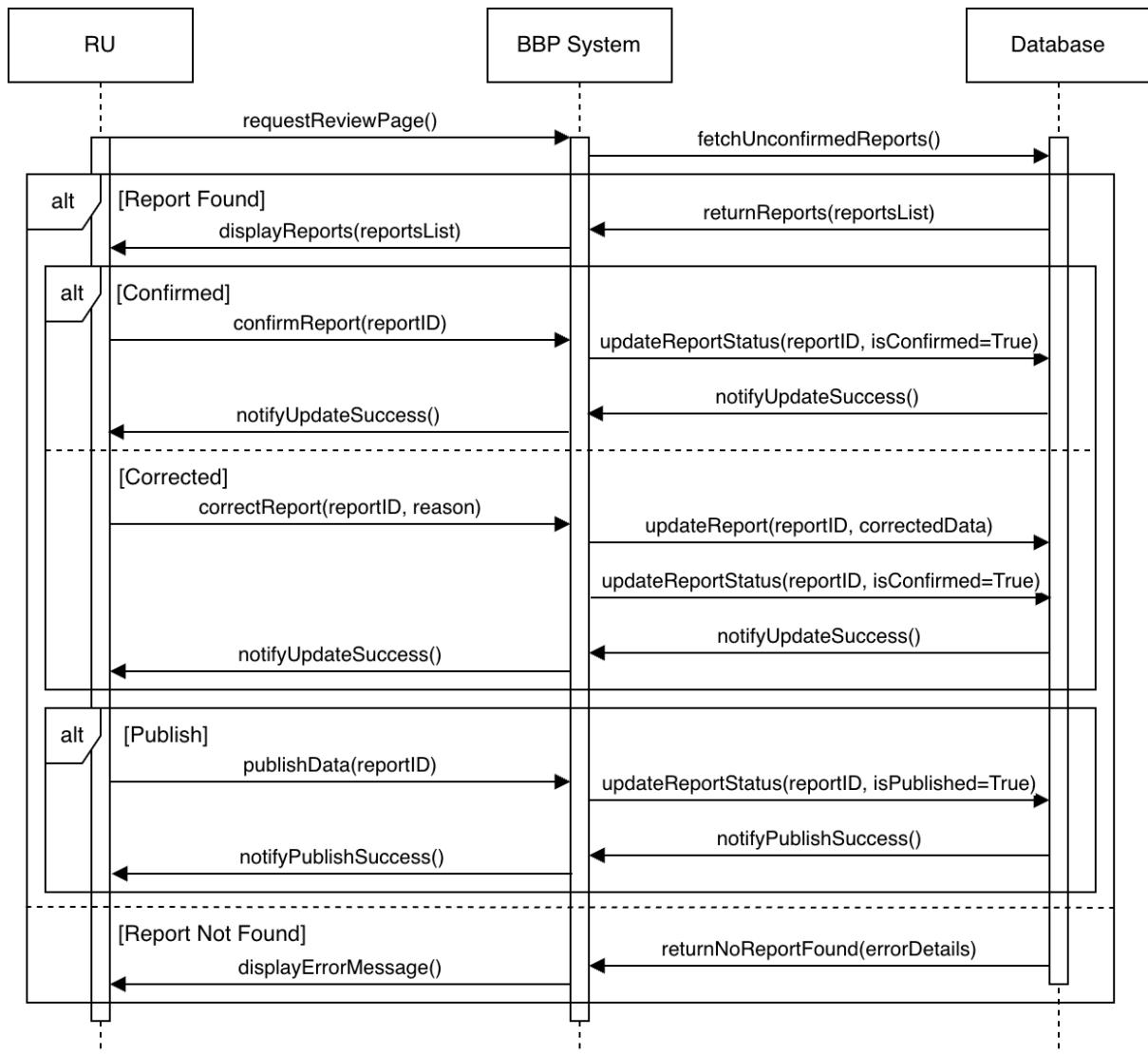


Figure 3.5: Review Auto-Detected Issues - Sequence Diagram

UCo4: Search and View Paths

This use case is common to both the Registered User and the Guest User.

Actors	User (RU or Guest), BBP System, Database, External Map Service (EMS)
Entry Conditions	The user is on the map search page.
Flow of Events	1. The user inputs an origin point and a destination. 2. BBP Systems asks EMS for routes. 3. BBP System searches for reports on the routes provided. 4. BBP System computes the PathScore internally. 5. BBP System requests the base map data from the EMS. 6. EMS returns the base map data. 7. BBP System displays the routes by overlaying its PathScore data onto the EMS map. 8. User selects a path.
Exit Conditions	The user has viewed the ranked path(s), including details of conditions and obstacles.
Exceptions	No Path Found: No valid bike path found between the two points

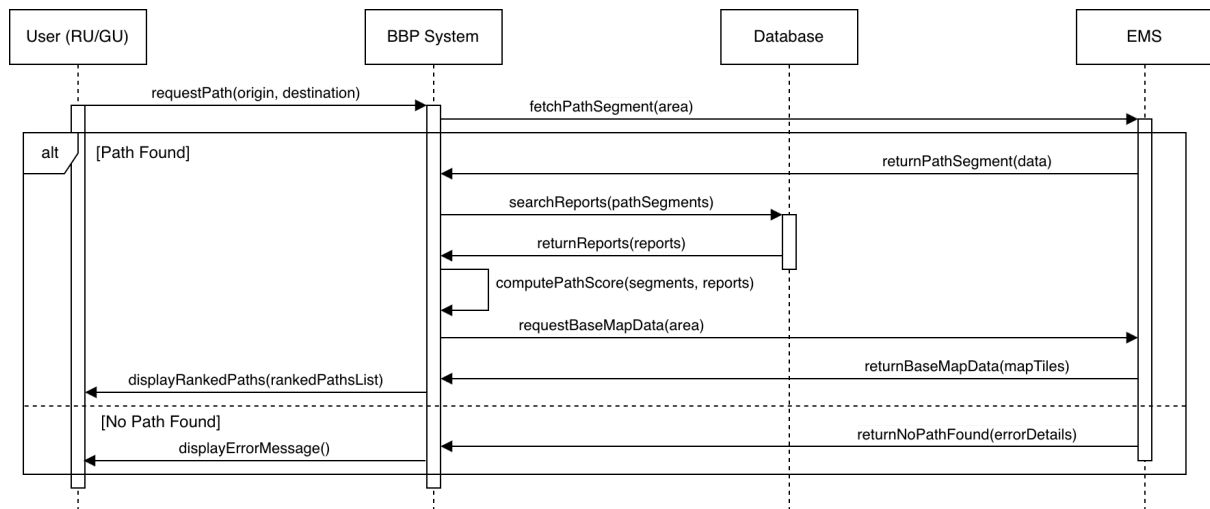


Figure 3.6: Search and View Paths - Sequence Diagram

UCo5: Manual Data Contribution

This use case covers the manual input of path status or obstacles by the RU.

Actors	Registered User (RU), BBP System, Database
Entry Conditions	RU is authenticated and on the data contribution page.
Flow of Events	1. RU selects the option "Manual mode" 2. RU specifies each streets of the path 3. RU enters status details for each street

	4. RU enters eventual obstacle type and description for each street 5. The BBP System check the adjacency of the streets in the path 6. BBP System sets the isConfirmed field and archives the new path with status/obstacles in the Database. 7. RU chooses to publish or draft the updated path 8. Database updates the isPublished field (if True, the data is now available to the community).
Exit Conditions	The new path with related conditions or obstacles is archived and validated. If published, the path information is available to the community.
Exceptions	Path Not Supported: Non-adjacent streets in the path

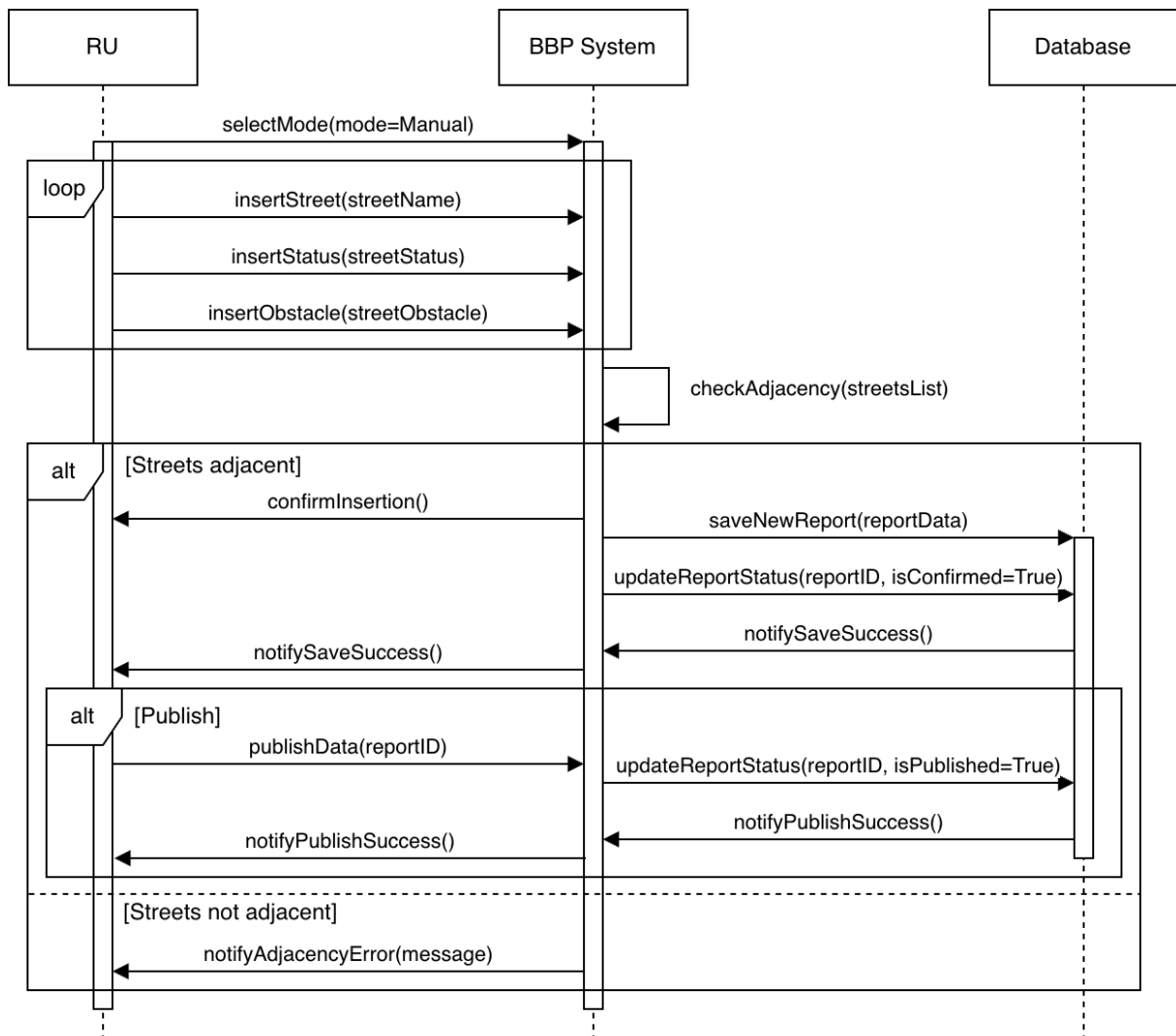


Figure 3.7: Manual Data Contribution - Sequence Diagram

UCo6: View Trip Details

This use case covers the management of the RU's private trip history.

Actors	Registered User (RU), BBP System, Database, External Map System (EMS)
Entry Conditions	The user must be logged in. At least one trip must exist in the system.
Flow of Events	1. User selects 'View Trips' or equivalent option. 2. System requests the list of existing trips from the Database/Storage. 3. System displays the list of trips. 4. User selects a specific trip (Trip ID: X). 5. System retrieves all detailed information for Trip X (including Map Data from the EMS). 6. System displays the Trip Details Screen: all data and the associated map.
Exit Conditions	The detailed information and map for Trip X are successfully displayed.
Exceptions	1. System Error: Unable to retrieve data from the Database 2. System Error: Unable to retrieve data from the EMS

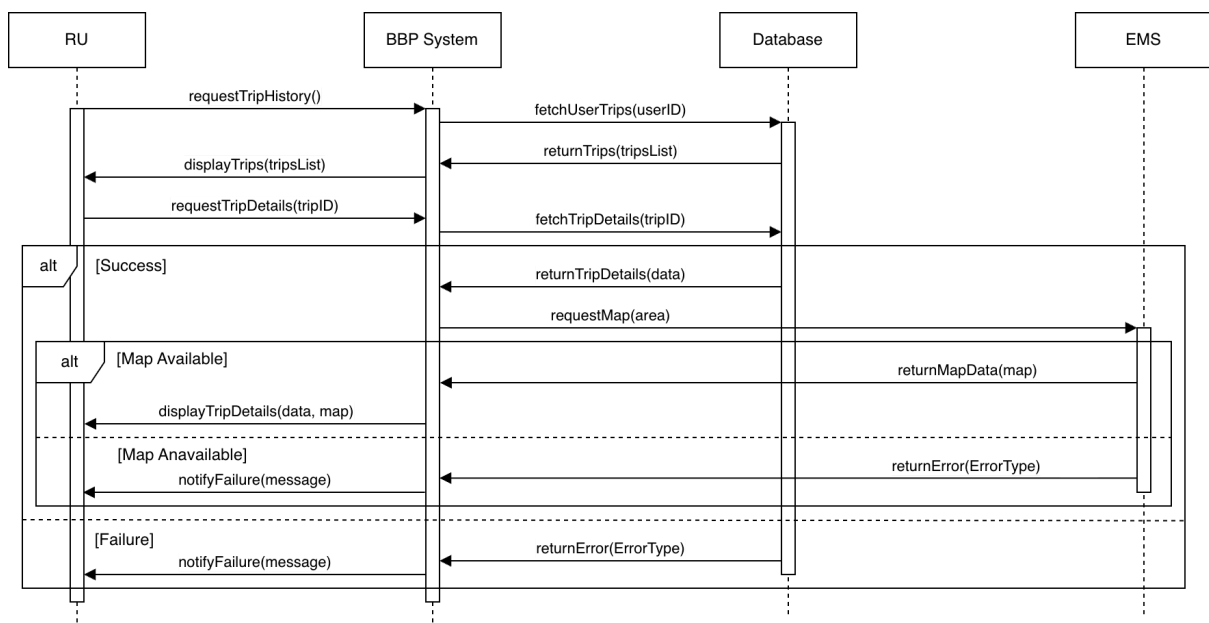


Figure 3.8: View Trip Details - Sequence Diagram

UCo7: Publish Validated Information

This use case handles the final action of making the data that has been validated and confirmed public.

Actors	Registered User (RU), BBP System, Database
--------	--

Entry Conditions	There are reports (Obstacle or SegmentStatus) that have been validated (isConfirmed=True) or created manually, but are not yet published.
Flow of Events	1. RU requests the list of "recorded and not published paths". 2. BBP System retrieves the list from the Database. 3. RU selects the data to be made public and confirms publication. 4. BBP System updates the data isPublished in the Database to True 5. BBP System notifies the RU of the successful publication.
Exit Conditions	The selected data is made public for the community
Exceptions	System Error: Failure to retrieve the unpublished list in the Database.

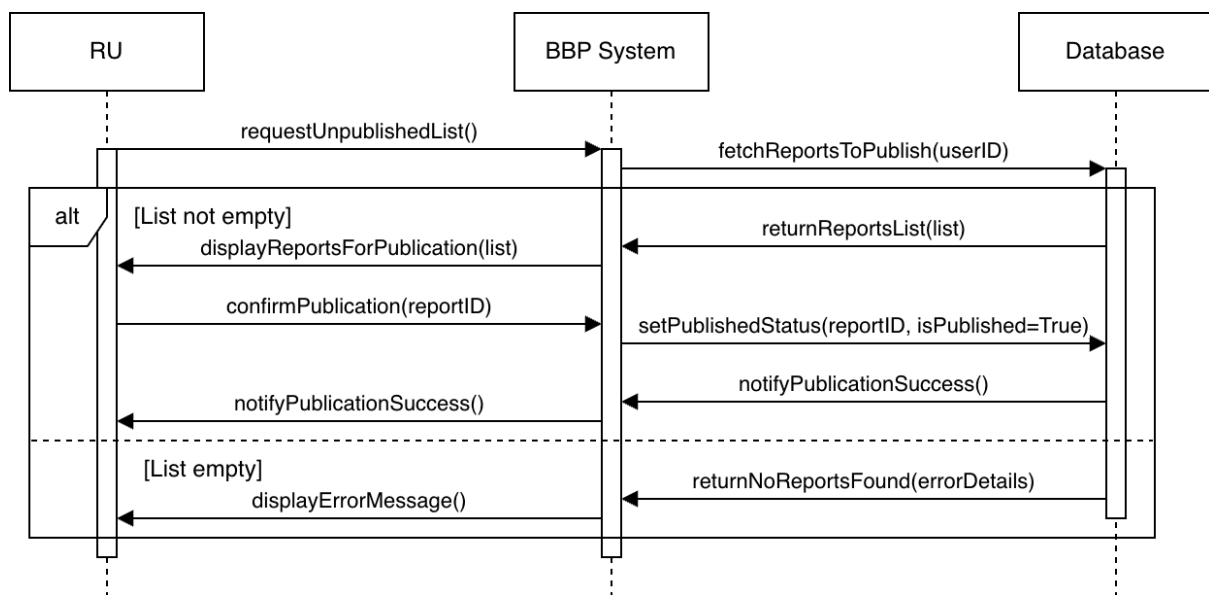


Figure 3.9: Publish Validated Information - Sequence Diagram

3.2.4. Goals - Requirements Mapping

G→	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
R01	✓														
R02		✓													
R03			✓												
R04				✓											
R05					✓										
R06						✓									
R07						✓									

R08							✓								
R09							✓								
R10		✓						✓							
R11								✓							
R12									✓						
R13										✓					
R14											✓				
R15												✓		✓	
R16													✓		✓
R17													✓		✓
R18														✓	✓
R19				✓								✓	✓	✓	✓

Table 3.2: Goals/Requirements Mapping table

3.3. Performance Requirements

- **Latency Requirements**

Regarding performance, BBP shall be developed with a scalable architecture capable of assuring its users a great Quality of Experience (QoE). To ensure fluid interaction, the latency between a user's action (e.g., initiating a path search, zooming the map, or confirming an automated report) and the system's corresponding visual or functional response shall not exceed 300 ms (excluding end-to-end transmission latency not attributable to the BBP system itself). Given the system's reliance on real-time routing, the computation and ranking of multiple paths between an origin and destination must be completed within a maximum of 2 seconds, ensuring timely planning and decision-making by both Registered and Guest Users.

- **Storage Requirements**

The system must be able to store the needed data safely and implement proper techniques of disaster recovery. The data to be stored is composed of user data, path geometry and large volumes of granular data such as GPS traces and SensorData (accelerometer/gyroscope). To maintain scalability and efficiency, the average storage footprint per Registered User, encompassing all private trips and associated data, should be maintained under 10 MB, utilizing appropriate compression techniques for data-intensive objects like GPS and sensor logs.

- **Architecture Sizing**

Assuming that one Italian out of ten daily use bikes and that, initially, BBP will be used by the 10% of the market capacity, the system architecture shall be sized to

accommodate a target user base of at least 600,000 Registered Users. Furthermore, the underlying infrastructure must be dynamically scalable, capable of automatically adjusting resources to accommodate peak traffic periods that may exceed twice the monthly average user count, ensuring long-term flexibility and stable performance.

3.4. Design Constraints

3.4.1. Standards compliance

The BBP system must adhere to legal and procedural standards to operate ethically and legally. Specifically, BBP shall be fully compliant with the European Union's General Data Protection Regulation (GDPR), which mandates stringent protection for all personal data collected, including trip history, location data and user identity, ensuring fundamental privacy rights are upheld. Furthermore, all communication involving the transmission of sensitive data, such as authentication credentials and location coordinates, must enforce the HTTPS protocol to ensure data security and integrity.

3.4.2. Hardware limitations

The system must maintain compatibility with standard, readily available technology. On the server side, BBP must be deployable on commodity x86 cloud infrastructure, avoiding reliance on proprietary or specialized hardware. On the client side, the application is constrained to run on common smartphones (Android/iOS) capable of installing and executing a mobile application and providing access to the required device sensors. In particular, the device must include a functional GPS receiver and motion sensors (accelerometer and gyroscope), since they are essential for core data collection and for supporting the Automated Mode.

3.4.3. Any other constraint

A critical constraint stemming directly from the project scope is the mandatory validation process: the BBP system must not publish any path information generated automatically by sensor data until it has been explicitly reviewed, confirmed or corrected by the Registered User. This is vital to preventing the dissemination of inaccurate data (false positives) that would undermine the system's core value proposition. Furthermore, BBP must respect the technical limitations and usage constraints of the External Weather Service (EWS) API and External Map Service (EMS) API used for trip data enrichment (e.g., rate limits, request quotas, and availability constraints).

3.5. Software System Attributes

3.5.1. Reliability

The BBP system must maintain a high level of reliability, especially concerning data collection, which forms the basis for the path score and the platform's overall utility. Absolute priority is the integrity of the raw data collected, such as GPS traces and

SensorData (accelerometer/gyroscope). The system shall guarantee data integrity for all recorded and stored trips. Trip data are primarily collected and persisted locally during recording and are synchronized with the backend only upon trip completion.

In case of temporary network unavailability, the system shall ensure that locally recorded data remain intact and are reliably uploaded once connectivity is restored, preventing data loss and preserving the correctness of the subsequent automatic validation process.

3.5.2. Availability

High availability is essential to support data acquisition and allow users across different time zones to plan routes. The BBP system shall be operational 24 hours a day, 7 days a week (24/7), with an annual uptime target of 99.5%, excluding planned maintenance periods. As the system relies on external services (EWS), integration must include fallback or caching mechanisms to prevent a temporary failure of the EWS from compromising the entire trip recording and archiving process.

3.5.3. Security

Given the sensitive nature of the data collected (geolocation, trip history), security is a critical non-functional attribute. The system shall use industry-standard cryptographic hashing methods for user password storage. All communications, both between the client and the server and between the server and external services (EWS, EMS), must be protected using the TLS 1.2+ protocol. Strict Role-Based Access Control (RBAC) shall be implemented to ensure that Guest Users can only access publicly published data, and Registered Users maintain privacy over their unpublished personal trips.

3.5.4. Maintainability

The system must be maintainable to facilitate future development, testing, and reduction of operational costs. The source code shall follow a standard coding style and utilize a modular architecture with a clear separation of concerns (e.g., between the PathScore logic and the user interface). Maintainability shall also be ensured by the presence of comprehensive unit and integration tests to guarantee that future modifications, particularly those related to the complex automatic review process, do not introduce regressions.

3.5.5. Portability

To guarantee maximum accessibility and implementation flexibility, the BBP system shall be designed for high portability. The backend architecture must be cloud-agnostic and make extensive use of containerization technologies, allowing efficient deployment across different cloud platforms (e.g., AWS, GCP, Azure).

On the client side, the user interface is implemented as a mobile application and must remain portable across the most common mobile operating systems (e.g., Android and iOS), ensuring consistent behavior and layout on a wide range of smartphone models and screen sizes. The application must rely on standard OS-provided APIs for GPS and

motion sensors (accelerometer, gyroscope) to reduce device-specific dependencies and maximize compatibility.

4. Formal Analysis using Alloy

This section presents the formal analysis of the Best Bike Paths (BBP) system using the Alloy language and the Alloy Analyzer tool.

The model focuses on the lifecycle of obstacle reports—both manually inserted and automatically detected—and on their impact on the path search and recommendation mechanism.

The goal of this analysis is to formally verify key invariants of the system, such as the correctness of the reporting workflow, the validation of automatically detected data and the selection of safe paths during route computation.

4.1. Alloy Model

The Alloy model abstracts users, path segments, obstacle reports, and search requests. The dynamic behavior of reports is modeled through a finite state machine, while path recommendation is constrained to avoid segments affected by published reports.

```
// Actors
sig RegisteredUser {}

// Map segments and paths
sig PathSegment {}

sig Path {
  segments : some PathSegment
}

// Subset of paths that the system recommends to the user
sig RecommendedPath in Path {}

// A path search request issued by a user (RU or GU)
sig SearchRequest {
  origin      : one PathSegment,
  destination : one PathSegment,
  candidatePaths: some Path,
  recommended  : set RecommendedPath
}

// Report states (temporal evolution of obstacle reports)
abstract sig ReportState {}
```

```

one sig AutoDetected, PendingValidation, Draft, Published
extends ReportState {}

// Report origin: manual insertion or automatic detection

abstract sig ReportMode {}
one sig Manual, Automatic extends ReportMode {}

// Obstacle / condition report on a specific path segment
sig ObstacleReport {
  segment : one PathSegment,
  author  : one RegisteredUser,
  mode    : one ReportMode,
  var state : one ReportState
}

// Facts of the model

// Static constraints on reports

// Manual reports are already validated: they never use
AutoDetected or PendingValidation
fact manual_reports_states {
  always all r: ObstacleReport |
    r.mode = Manual implies r.state in Draft + Published
}

// Automatic reports may pass through all the states of the FSM
fact automatic_reports_states {
  always all r: ObstacleReport |
    r.mode = Automatic implies
      r.state in AutoDetected + PendingValidation + Draft +
Published
}

// Published reports must always have an author

```

```

fact published_has_author {
  always all r: ObstacleReport |
    r.state = Published implies some r.author
}

```

The evolution of obstacle reports is modeled as a finite state machine. Only forward transitions are allowed, preventing reports from skipping mandatory validation steps or reverting to previous states.

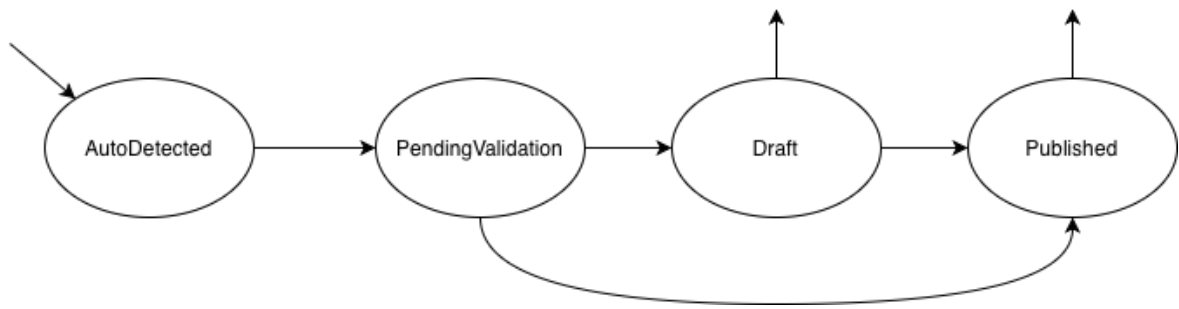


Figure 4.1: Report FSM - Alloy Model

```

// Temporal evolution (FSM of reports)

// Legal forward transitions only
fact evolution {
  always
  all r: ObstacleReport |
    r.mode = Automatic and r.state = AutoDetected
    implies r.state' = PendingValidation

  always all r: ObstacleReport |
    r.state = PendingValidation
    implies (r.state' = Draft or r.state' = Published)

  always all r: ObstacleReport |
    r.state = Draft implies (r.state' = Draft or r.state' =
    Published)

  always all r: ObstacleReport |
    r.state = Published implies r.state' = Published
}

```

```

}

// No backward transitions between states
fact no_backward_transitions {
  always all r: ObstacleReport |
    (r.state = Draft implies
      r.state' != PendingValidation and r.state' !=
AutoDetected) and
    (r.state = Published implies r.state' = Published) and
      (r.state = PendingValidation implies r.state' !=
AutoDetected)
}

// No illegal direct jumps: automatic reports cannot skip
validation
fact no_illegal_jumps {
  always all r: ObstacleReport |
    (r.state = AutoDetected implies
      r.state' != Draft and r.state' != Published)
}

// Every report eventually stabilizes in Draft or Published
fact report_eventually_draft_or_published {
  all r: ObstacleReport |
    eventually r.state in Draft + Published
}

```

The model also includes a simplified representation of path search requests. Each search connects an origin and a destination through a set of candidate paths. Only published reports are considered when evaluating paths, and recommended paths must avoid segments affected by published obstacles whenever possible.

```

// Constraints about path search & generation

// Each candidate path in a search must connect origin and
destination
fact search_paths_connect_extremes {
  all sr: SearchRequest |

```

```

    all p: sr.candidatePaths |
        sr.origin in p.segments and sr.destination in p.segments
}

// Recommended paths for a search are a subset of candidate
paths
fact recommended_subset_candidate {
    all sr: SearchRequest |
        sr.recommended in sr.candidatePaths
}

// if there are two segments, one with an obstacle report and
one without, choose the second
fact recommend_only_safe_when_possible {
    all sr: SearchRequest |
        (some p: sr.candidatePaths | not pathAffected[p]) implies (
            all rp: sr.recommended | not pathAffected[rp]
            and some sr.recommended
        )
}

fact origin_destination_distinct {
    all sr: SearchRequest | sr.origin != sr.destination
}

// Helper functions and predicates

// Reports that are active for routing on a segment: only
Published reports
fun activeReports[s: PathSegment]: set ObstacleReport {
    { r: ObstacleReport | r.segment = s and r.state = Published }
}

pred init {
    all r: ObstacleReport |
        (r.mode = Automatic implies r.state = AutoDetected) and
        (r.mode = Manual implies r.state = Draft)
}

```

```

}

fact initial_state { init }

// A path is affected if it traverses at least one segment with
a Published report
pred pathAffected[p: Path] {
  some s: p.segments | some activeReports[s]
}

// Set of reports visible along a given path (those Published on
its segments)
fun visibleReportsOnPath[p: Path]: set ObstacleReport {
  { r: ObstacleReport |
    r.state = Published and some s: p.segments | r.segment = s
  }
}

// Does the system contain both manual and automatic reports?
pred mixed_origin_reports {
  some r1, r2: ObstacleReport |
    r1.mode = Manual and r2.mode = Automatic
}

// Has an automatic report followed the full validation flow?
// (AutoDetected → PendingValidation → Draft/Published)
pred automatic_report_validated[r: ObstacleReport] {
  r.mode = Automatic and
  once r.state = AutoDetected and
  once r.state = PendingValidation and
  eventually r.state in Draft + Published
}

// Assertions about correctness of the model

// Automatic reports that end up Published must have passed
through PendingValidation

```

```

assert automatic_reports_must_be_validated {
  all r: ObstacleReport |
    r.mode = Automatic and r.state = Published
    implies once r.state = PendingValidation
}

// For each search request, if there exists at least one
// candidate path that is not affected by Published reports, then
// all recommended
// paths in that search must avoid affected segments
assert recommended_avoid_when_possible {
  all sr: SearchRequest |
    (some p: sr.candidatePaths | not pathAffected[p]) implies
      all rp: sr.recommended | not pathAffected[rp]
}

// Draft reports are private and must not affect routing
// decisions
assert draft_reports_do_not_affect_routing {
  all p: Path |
    (some r: ObstacleReport | r.state = Draft and r.segment in
p.segments) and
    (no r: ObstacleReport | r.state = Published and r.segment
in p.segments)
    implies not pathAffected[p]
}

// Predicates used to generate instances

// show an instance with:
// - at least one automatic report that follows the full flow
//   (AutoDetected → PendingValidation → Draft → Published)
// - at least one manual report that eventually becomes
//   Published
pred showExample {
  init

```

```

some ra, rm: ObstacleReport |
  ra.mode = Automatic and
  eventually ra.state = Draft and
  eventually ra.state = Published and
  rm.mode = Manual and
  eventually rm.state = Published
}

// Example of path search where two paths connect origin and
// destination:
// one path contains a Published obstacle report and is
// discarded,
// the other is clean and therefore recommended.
pred showPaths {
  eventually some sr: SearchRequest |
    some pBad, pGood: Path |
      pBad in sr.candidatePaths and
      pGood in sr.candidatePaths and
      pBad != pGood and
      pathAffected[pBad] and
      not pathAffected[pGood] and
      pGood in RecommendedPath and
      sr.recommended = pGood
}

```

4.2. Alloy Results

The model has been tested using the Alloy Analyzer tool. No counterexamples were found within the explored scopes to the assertions.

The following results are obtained executing the `pred showExemple` and represent the correct temporal flow of a reporting process for both automatic and manual mode (respectively `ObstacleReport0` and `ObstacleReport1`).

Time 0: AutoDetected, Draft

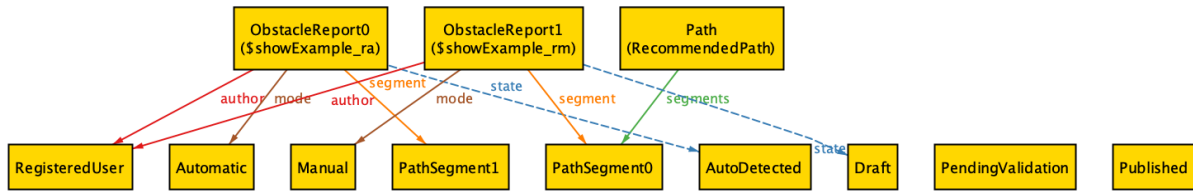


Figure 4.2: AutoDetected State - Alloy Result

Time 1: PendingValidation, Draft

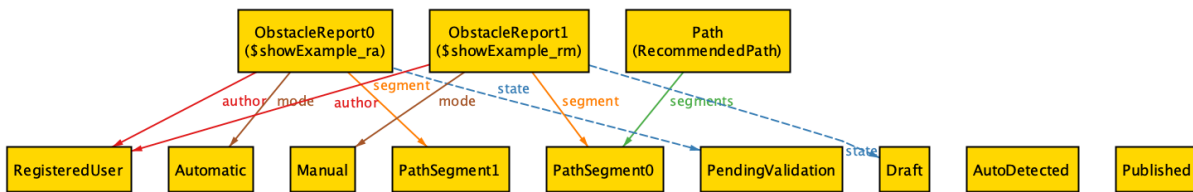


Figure 4.3: PendingValidation State - Alloy Result

Time 2: Draft(Validated), Draft

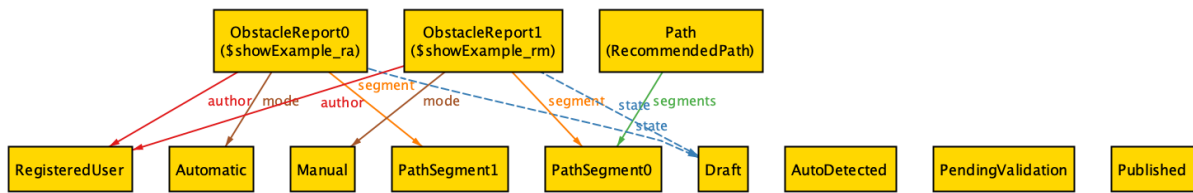


Figure 4.4: Draft State - Alloy Result

Time 3: Published, Published

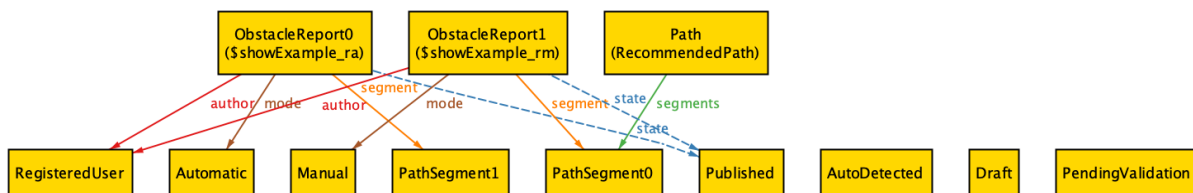


Figure 4.5: Published State - Alloy Result

The following results are obtained executing the pred showPaths and represent the correct process to generate a path using a published report, this means that if two segments (origin and destination) are connected by other two segments, the BBP will recommend the one with no ObstacleReport as the best one.

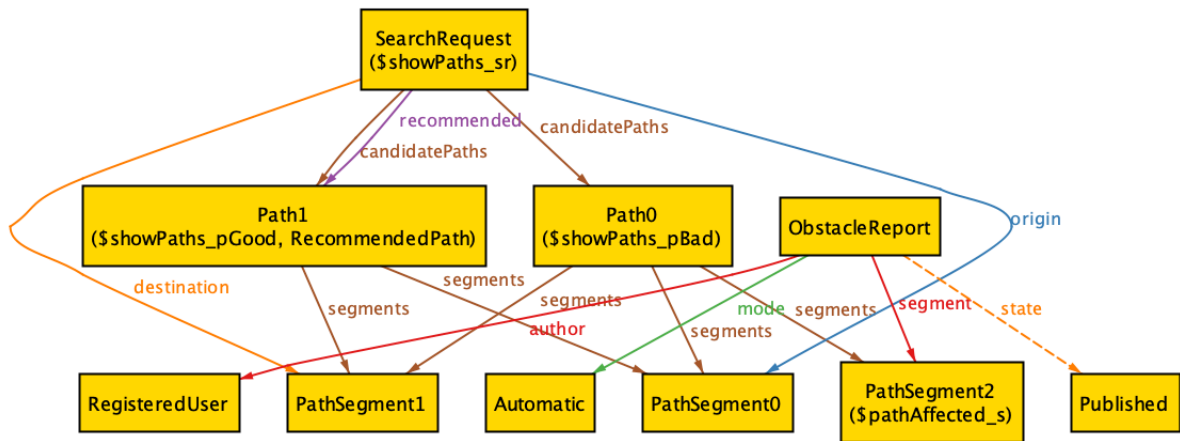


Figure 4.6: SearchRequest - Alloy Result

5. Effort spent

Group Member	Effort Spent	
Lorusso Luca (10831296)	Introduction	1 h
	Overall Description	7 h
	Specific Requirements	15 h
	Formal Analysis	7 h
	Reasoning	5.5 h
Martorano Gabriele (10822443)	Introduction	2 h
	Overall Description	13 h
	Specific Requirements	8 h
	Formal Analysis	8 h
	Reasoning	4.5 h

Table 5.1: Member Effort Spent table

6. References

6.1. References

- Assignment RDD AY 2025-2026
- github - LorussoMartorano

6.2. Used tools

- GitHub – Used for version control and collaborative development.
- draw.io – Used for UML, architectural and deployment diagrams.
- Alloy Analyzer – Used for formal modeling and verification of system properties.
- ChatGPT:
 - **Inputs provided:** Draft sections of the RASD.
 - **Outputs obtained:** Suggestions for text refinement, summarization, and improved consistency of descriptions.
 - **Verification:** All generated content was manually reviewed, validated, and integrated by the authors to ensure correctness and coherence with the project specification.